

Testing Deep Reinforcement Learning

September 2026

Why testing DRL is different

- A conventional DNN is often tested as a mapping

$$x \longrightarrow y$$

with a known or approximated expected output.

- An RL agent instead produces a *sequence*:

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1} \rightarrow a_{t+1} \rightarrow \dots$$

- Failures can arise from the policy, environment interaction, exploration, reward/cost implementation, or training algorithm.
- Therefore, DRL testing needs both:
 - ① coverage criteria — what has been exercised?
 - ② test oracles — what counts as incorrect?

A useful testing taxonomy

Layer	Question
Implementation	Did the SAC/CSAC code execute the relevant logic?
Neural network	Did tests exercise diverse internal network behavior?
Environment	Which parts of the state/transition space were exercised?
Policy	Which decisions and state–action behaviors were exercised?
Trajectory	Which sequential behaviors were exercised?
Constraint	Which safe/unsafe regions and budget behaviors were exercised?
Oracle	How do we determine whether behavior is correct?
Test generation	How do we systematically produce useful test cases?

Implementation-based coverage

- Conventional software testing:
 - statement coverage
 - branch coverage
 - function/method coverage
 - path coverage
- Useful for validating the *implementation* of SAC/CSAC.
- Examples:
 - actor update executed
 - critic update executed
 - target-network update executed
 - entropy-temperature update executed
 - CSAC cost-critic and Lagrange updates executed

Limitation: high code coverage does not imply adequate coverage of the MDP or learned policy.

Neural-network coverage

- DNN testing introduced coverage criteria based on neuron activations.
- Basic neuron coverage:

$$C_{\text{neuron}} = \frac{\text{\#activated neurons}}{\text{\#neurons}}$$

- More granular criteria include:
 - k -multisection coverage
 - strong activation coverage
 - boundary coverage
 - top- k coverage
 - neuron-pattern coverage
- Relevant to actor/critic networks, but not sufficient for DRL system adequacy.

State / environment coverage

- RL is fundamentally an MDP:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R)$$

- State coverage asks: which regions of $\mathcal{S}$ have been exercised?
- For discrete path planning, states can be partitioned by:
 - agent position
 - obstacle configuration
 - distance to goal
 - boundary/dead-end status
 - constraint budget/state
- Coverage can be exact for small discrete spaces or partition/feature-based for larger spaces.

Action coverage

For a discrete action space:

$$\mathcal{A} = \{a_1, \dots, a_n\}$$

$$C_A = \frac{|\{a : a \text{ selected by tests}\}|}{|\mathcal{A}|}$$

- Simple and cheap.
- Useful for detecting completely unexercised actions.
- Particularly natural for discrete SAC.
- Weak criterion by itself:
 - selecting every action once says little about *where* it was selected;
 - it does not capture action probabilities or temporal context.

State–action / policy coverage

A policy maps states to decisions:

$$\pi(a \mid s)$$

For deterministic policies:

$$(s, a) \in \mathcal{S} \times \mathcal{A}$$

For stochastic policies, the full distribution matters:

$$\pi(\cdot \mid s)$$

- State–action coverage asks which decisions were exercised in which states.
- Policy coverage can capture the distribution or partition of actions over states.
- This is substantially closer to testing the actual RL behavior than neuron coverage.

Trajectory coverage

A test case is naturally a trajectory:

$$\tau = (s_0, a_0, r_0, s_1, a_1, \dots, s_T)$$

Useful trajectory classes for path planning:

- reaches goal
- collision
- boundary hit
- dead end
- cycle / repeated-state behavior
- timeout
- constraint violation
- successful but unnecessarily long path

Constraint coverage

For constrained RL, define safe and unsafe regions:

$$g(\tau) \leq d$$

where $g(\tau)$ is accumulated constraint cost and d is the allowed budget.

Possible coverage dimensions:

- safe states
- near-boundary states
- constraint-violating states
- different violation magnitudes
- budget depletion levels
- transitions into/out of unsafe regions

CSAC-specific importance: constraint behavior should be tested independently from reward performance.

The test-oracle problem

In supervised learning:

$$x \rightarrow y^*$$

often provides a direct oracle.

In RL:

$$s_0 \rightarrow a_0 \rightarrow s_1 \rightarrow \dots$$

usually has no unique correct action.

Possible oracles therefore include:

- hard safety properties
- reward/performance thresholds
- constraint satisfaction
- known optimal solutions
- mathematical implementation invariants

Reward-based oracles

For a path-planning episode:

$$J_R(\tau) = \sum_{t=0}^T \gamma^t r_t$$

Possible checks:

- minimum acceptable return
- goal-reaching rate
- path-length threshold
- regret / optimality gap when an optimum is known
- degradation relative to a reference policy

But:

high reward $\nRightarrow$ correct implementation

A reward function can itself be incorrectly implemented, or reward shaping can hide undesirable behavior.

Safety and constraint oracles

For path planning, many properties can be exact:

$$\text{collision count} = 0$$

$$s_t \notin \mathcal{S}_{\text{forbidden}}$$

$$g(\tau) \leq d$$

$$\exists t : s_t = s_{\text{goal}}$$

These are stronger oracles than statistical reward thresholds.

For CSAC:

$$J_C(\pi) \leq d$$

should be evaluated separately from:

$$J_R(\pi)$$

This creates a natural two-dimensional evaluation:

(performance, constraint satisfaction)

Mutation testing

Instead of asking only whether tests execute code, inject realistic faults.

Example SAC mutants:

- wrong entropy sign
- incorrect discount factor
- incorrect Bellman target
- wrong target-network update
- actor/critic parameter mismatch
- incorrect action masking
- wrong cost accumulation
- wrong constraint threshold
- incorrect λ update
- reward/cost critic mix-up
- incorrect constraint penalty sign

Mutation score:

$$MS = \frac{\text{\#detected mutants}}{\text{\#non-equivalent mutants}}$$

Example: 5×5 constrained grid path planning

S	.	.	#	.
.	#	.	#	.
.	#	.	.	.
.	.	.	#	.
.	.	.	.	G

$$\mathcal{A} = \{U, D, L, R\}$$

State

$$s_t = (r_t, c_t), \quad \mathcal{S} = \{\text{free cells}\}$$

20 free states.

Transition

$$P(s'|s, a) = \begin{cases} 0.9 & s' = T(s, a) \\ 0.1 & s' = s \end{cases}$$

for valid moves; invalid moves remain in place.

Reward

$$R = \begin{cases} +100 & \text{goal} \\ -5 & \text{invalid/collision attempt} \\ -1 & \text{otherwise} \end{cases}$$

CSAC constraint

$$J_C(\pi) = \mathbb{E}_\pi \left[\sum_t C_t \right] \leq d, \quad d = 2.$$

Coverage criteria

Criterion	Definition	What it tells us
Code coverage	Executed statements/branches	Which SAC/CSAC implementation logic ran?
Neuron coverage	Activated actor/critic neurons	Which internal NN regions were exercised?
State coverage	$\frac{ S_{\text{visited}} }{ S }$	Which MDP states were visited?
Action coverage	$\frac{ A_{\text{selected}} }{ A }$	Were U, D, L, R all exercised?
State-action coverage	$\frac{ (s,a)_{\text{visited}} }{ S A }$	Which policy decisions were exercised?
Transition coverage	Visited (s, a, s') combinations	Which environment dynamics were exercised?
Trajectory coverage	Classes of trajectories/outcomes	Which sequential behaviors were exercised?

3. Which coverage criteria matter for SAC/CSAC?

Criterion	SAC	CSAC	Interpretation
Code/branch	✓	✓	Necessary implementation testing; not behavioral adequacy.
Neuron	~	~	Diagnostic; weak connection to MDP behavior.
State	✓✓	✓✓	Strong when the state space is finite/discretized.
Action	✓	✓	Useful but weak alone.
State-action	✓✓	✓✓	Directly exercises policy decisions.
Transition	✓✓	✓✓	Tests policy-environment interaction.
Trajectory	✓✓	✓✓	Captures sequential failures.
Policy entropy	✓✓	✓✓	Especially relevant to entropy-regularized SAC.
Constraint	—	✓✓	Essential: safe, boundary and violating behavior.

Coverage $\neq$ correctness

Designing test oracles

A coverage criterion answers “*what did we exercise?*” An oracle answers “*was the behavior correct?*”

Oracle	Example	Use
Environment	$s' = T(s, a)$	Detect incorrect transition/environment implementation.
Safety	$\text{collision} = 0$	Hard correctness property.
Goal/reachability	$\exists t : s_t = s_G$	Check successful completion.
Performance	$L(\tau) - L^*$	Compare against known optimal path.
Reward	$J_R \geq J_{\min}$	Regression/performance testing.
Constraint	$J_C \leq d$	Essential CSAC safety oracle.
Algorithmic	$\sum_a \pi(a s) = 1$	Implementation invariant.